ISYE 7406 – Homework 5

Jonathan Feng

Introduction

In this homework, we were tasked with choosing our own dataset and applying two methods to the dataset (random forest, boosting). From what we have learned in our course we will apply the proper data cleaning, exploratory analysis, parameter selection, and tuning in these runs in tandem with some baseline models like logistic regression and decision trees.

Problem Statement & Data Source

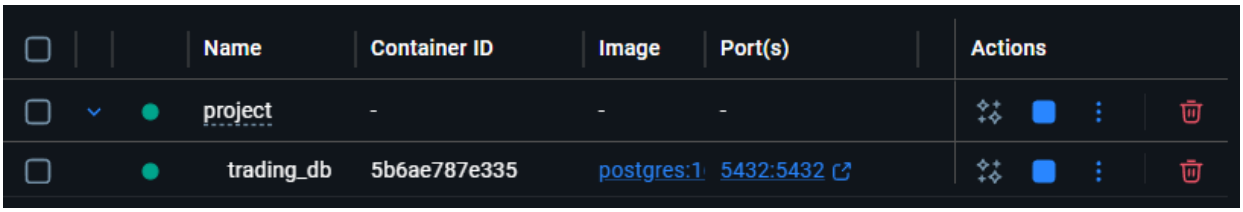
For the data source, I ended up using alpaca's api for historical stock data since it was related to data used in my final project. I always found algo or day trading interesting, but I've never attempted it myself. It seems super daunting due to my lack of knowledge in the domain, but I thought it would be simple enough to try and do a "dumb trading" based off next day direction of the stock (up or down).

This led to the problem statement being defined as: whether or not you could use the base historical data, OHLC (open, high, low, close), additional data (vwap, volume, trade_count), and engineered features that are more relevant to return like (log return, high low range, volume ratio, rolling volume, and rolling sharpe) to predict the next day direction of the stock. Based on the statement, 1813 rows of daily data across these 13 features were collected and analyzed over the course of an interesting period (2019-2026).

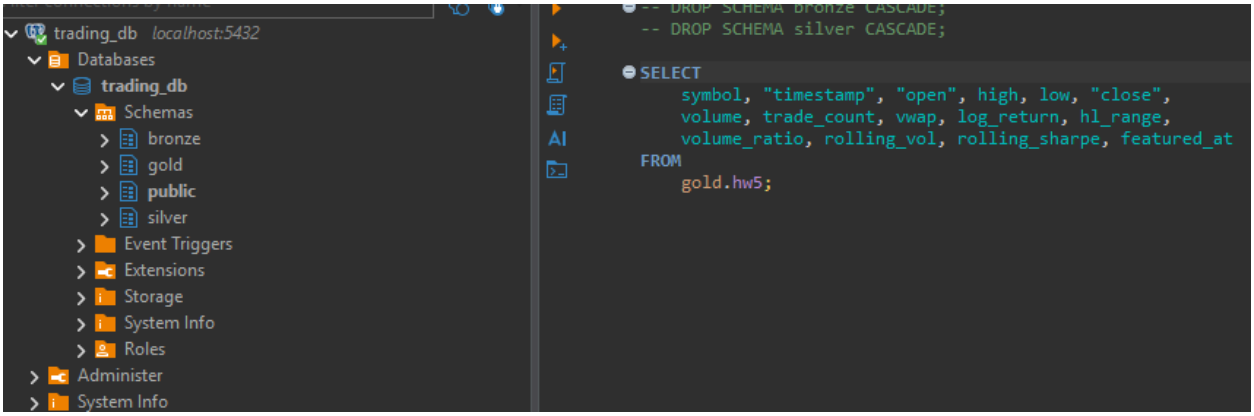
Methodology

The methodology follows a very simple exploratory data analysis, data cleaning, feature selection, modeling, cross validation, and metrics structure. However, I set up an additional few steps in the acquisition of the data because I wanted this to be reusable for future algorithm trading. The additional setup consists of (api -> bronze -> silver -> gold) pipeline, prior to entering the main methodology with set schemas, data quality checks, first time bulk data processing, quarantine zone for bad data, silver transformations, and incremental ingestion logic. This is done via a persistent storage Postgres database, spun up with a docker container. The daily DAG/pipeline run is triggered via windows task scheduler along with the associated power shell script.

Post data gathering, polars library pulls my gold table into the jupyter notebook used for this homework. An ml-flow instance is also spun up for model tracking. Below are a few pictures of the setup if the reader is curious.



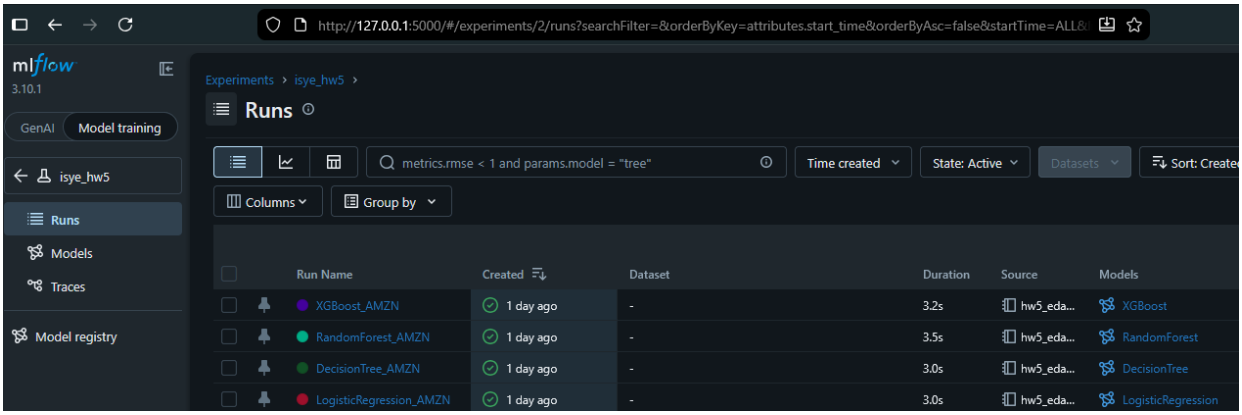
Postgres Container



Medallion Architecture

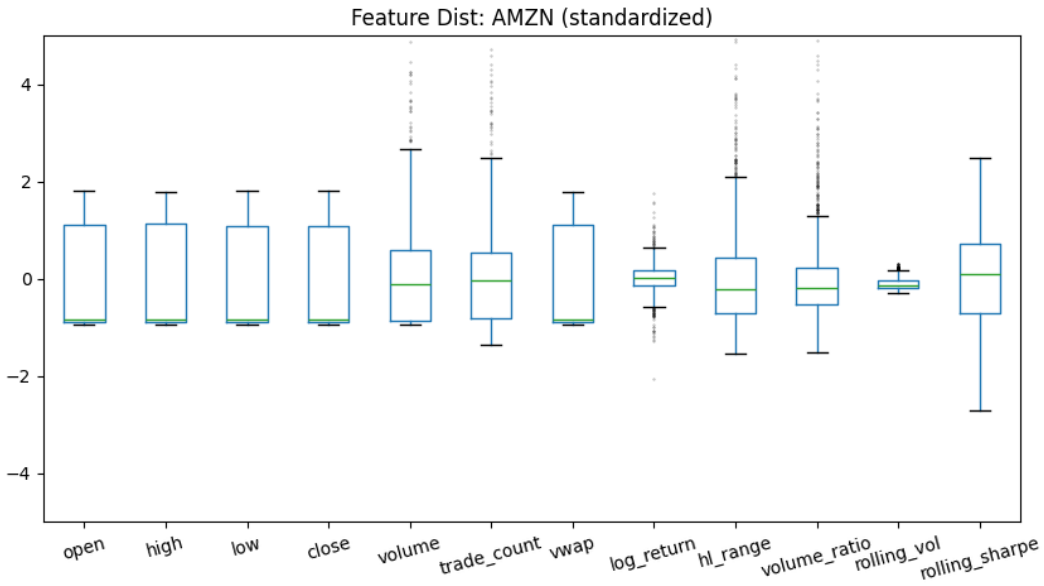
	symbol	timestamp	open	high	low	close	volume	trade_count	vwap
1	AMZN	2019-01-01 21:00:00.000 -0800	1,465.2	1,553.36	1,460.93	1,539.13	8,279,074	196,916	1,52
2	AMZN	2019-01-02 21:00:00.000 -0800	1,520.01	1,538	1,497.11	1,500.28	7,199,259	172,823	1,51
3	AMZN	2019-01-03 21:00:00.000 -0800	1,530	1,594	1,518.3101	1,575.39	9,555,309	218,192	1,56
4	AMZN	2019-01-06 21:00:00.000 -0800	1,602.31	1,634.56	1,589.185	1,629.51	8,302,242	189,817	1,61
5	AMZN	2019-01-07 21:00:00.000 -0800	1,664.69	1,676.61	1,616.61	1,656.58	9,159,499	227,576	1,65
6	AMZN	2019-01-08 21:00:00.000 -0800	1,652.98	1,667.7989	1,641.3953	1,659.42	6,586,395	160,674	1,65
7	AMZN	2019-01-09 21:00:00.000 -0800	1,641.01	1,663.25	1,621.615	1,656.22	6,733,620	151,501	1,6
8	AMZN	2019-01-10 21:00:00.000 -0800	1,640.55	1,660.294	1,636.22	1,640.56	4,893,903	114,125	1,64
9	AMZN	2019-01-13 21:00:00.000 -0800	1,615	1,648.2	1,595.15	1,617.21	6,304,455	125,684	1,62
10	AMZN	2019-01-14 21:00:00.000 -0800	1,632	1,675.16	1,626.01	1,674.56	6,216,788	139,660	1,65
11	AMZN	2019-01-15 21:00:00.000 -0800	1,684.22	1,705	1,675.8787	1,683.78	6,662,697	137,719	1,69

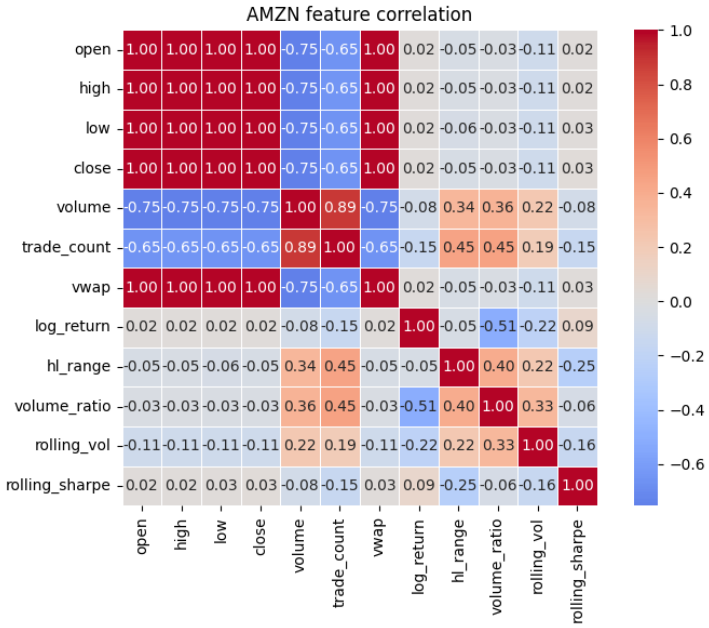
DBeaver Query Results



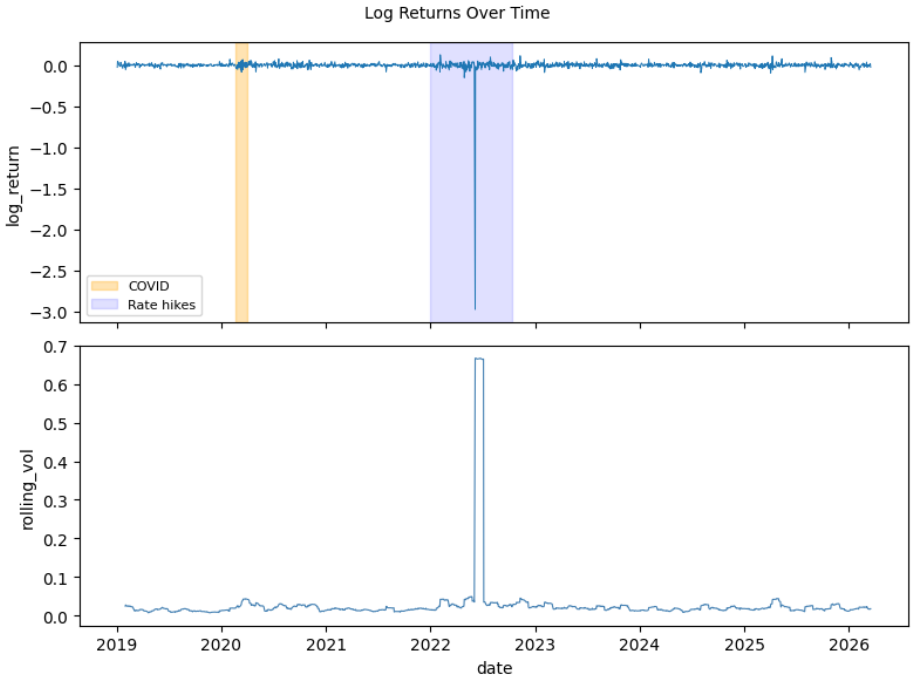
Mlflow Web UI

After the setup, exploratory data analysis is performed for the distribution checking, outlier analysis, target variable spread, correlation heat map, and log returns chart. Based on the EDA results, I have selected suitable variables with respect to correlation and since they are returns related. The correlation of most of the original variables was also near perfect due to the price levels moving together. If I had included them within the model, it would introduce high multicollinearity.





In general, the outliers look acceptable being lower than 2% of the actual dataset, and the spread of the target variable 0 being down and 1 being up was near even, so we knew that our data was not skewed towards any direction. However, I was very curious about how the log returns looked over time. I think it was perfect for our case due to the date range capturing a very interesting time in economic history (covid + rate hikes). Aside from the spikes in expected areas, you can see that the log return is relatively consistent.



For the modeling process, I have chosen to go with decision tree and logistic regression for my base models. The necessary models of random forest and boosting will have their own cross validation method and utilize time series with 5 splits. RF and boost hyper parameters are a general spread and selected via the 5-fold time series split during grid search cross validation. The data is split around 2025-01-01 for convenience on the training and testing set, and the score used in this selection is ROC AUC.

Results and Analysis

After implementing the modeling and cross validation on the 4 models chosen, I have a function that tables and visualizes most of the results via mlflow. The results that are displayed for accuracy and roc_auc are pretty much the same. All land around the 0.5 mark with decision trees performing slightly better at 0.53, and 0.54 and the others varying slightly by 0.1 give or take.

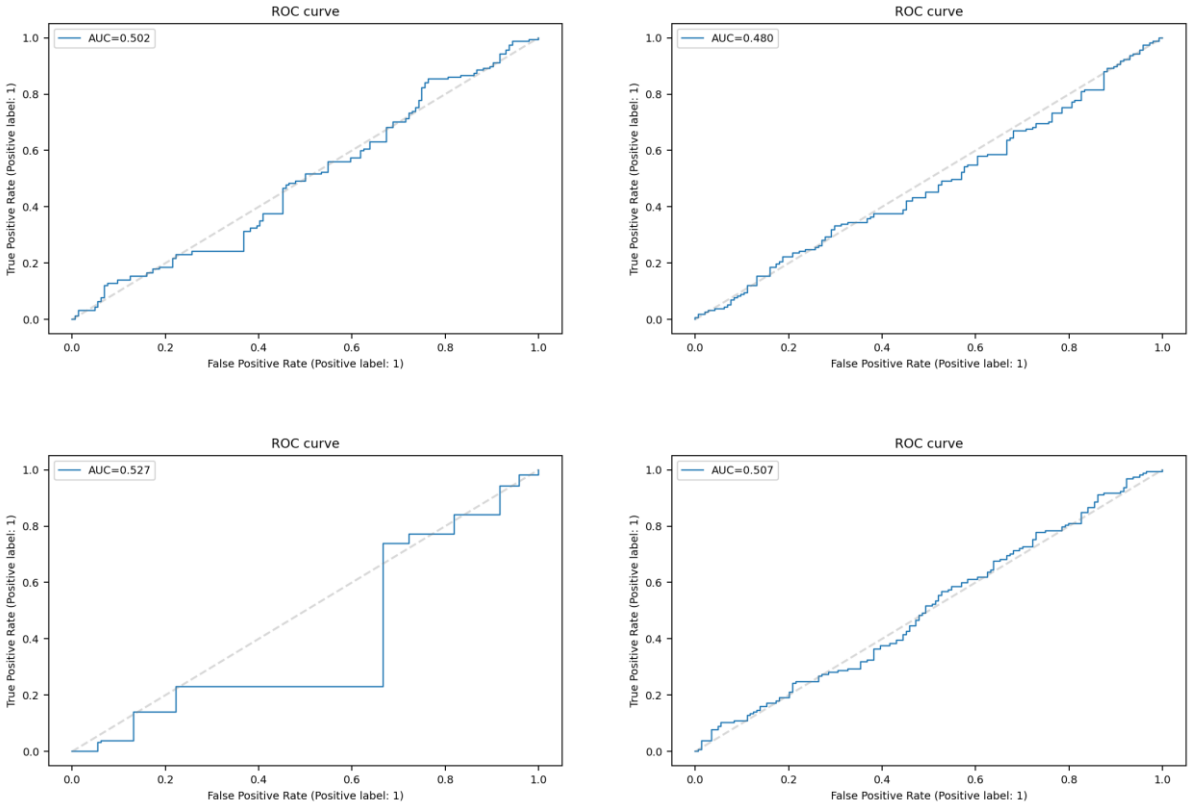
For model reference (grey:xgboost, blue:random forest, green: decision tree, red: logistic regression).

The explanation is that it is basically no better than a coin flip, with AUC references as 0.5 for a random baseline and 1 being a perfect score. For more context, the final parameters chosen after grid search: for xgboost were (learning_rate: 0.01, max_depth: 3, n_estimators: 100) and (max_depth:5, max_features: sqrt, n_estimators: 200) for random forest. I believe the interpretations of these selected parameters can be due to slow learning rate (model being cautious about over fitting), selected a shallower tree to also prevent overfitting, and a reasonable standard for estimators. A similar pattern was followed but for diversity between trees on max features and more trees for stability.

I realized that for these default parameters, there was not much to choose between, but given more time I would train more than just a couple of parameters with grid search. However, I believe that even with more to choose from, the outcome could smooth out at some point due to the limited set of features.

Below I have also included the roc curve charts. A quick analysis yields that xg boost is essentially random, there is no meaningful signal. Random forest exhibits a score slightly below random, which is worse than 50:50 odds itself, the base models are marginally above random but still essentially flat. The linear boundary from logistic regression can't separate up and down days easy, and decision trees actually performs the best out of the 4. You can see the noticeable step pattern as its finding a couple splits that capture the signal but not consistently enough overall.

The plots below are in this order left to right and wrapping around (xgboost, random forest, decision tree, logistic regression).



Conclusion

Overall, the decision tree baseline model performed the best out of the 4 models trained (xgboost, random forest, decision tree, and logistic regression) for the stock ticker AMZN. For a simple up and down price prediction for next day trading, However, all of them perform close to the baseline 0.5 which is considered random and is consistent with the concept of weak form market efficiency. The daily prediction of direction is largely unpredictable from generic technical features that are engineered. At the end of the day, a simpler model would probably be best in my case when there isn't enough to learn off of it. For future direction, more thoughtful feature engineering needs to be put in place in tandem with more variety in cross validation parameters.

Appendix

```
from dotenv import load_dotenv
import polars as pl
import pandas as pd
import os
import mlflow
import mlflow.sklearn
from mlflow.tracking import MlflowClient
from datetime import date
import scipy.stats as stats
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from scipy.stats import zscore
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix
import xgboost as xgb
import mlflow, mlflow.sklearn
import os

# load global variables
load_dotenv()

DB_URL =
f"postgresql://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}@localhost:5432/{os.getenv('POSTGRES_DB')}"
GOLD_TABLE=os.getenv("GOLD_TABLE")

# read in data
query = f"""
    SELECT g.* FROM {GOLD_TABLE} as g
"""

gold_df = pl.read_database_uri(
    query=query,
    uri=DB_URL
)

# filter for each ticker
amzn_df = gold_df.filter(
    pl.col("symbol") == "AMZN"
)

xlk_df = gold_df.filter(
    pl.col("symbol") == "XLK"
)

# columns to_drop
drop_cols = ['symbol', 'featured_at']
amzn_df = amzn_df.drop(drop_cols)
xlk_df = xlk_df.drop(drop_cols)

# sanity check
print(f'AMZN Rows Detected: {len(amzn_df)}')
print(f'XLK Rows Detected: {len(xlk_df)}')
print(f'Columns: {amzn_df.columns}')
print(f'Dtypes:\n{amzn_df.schema}')
print(f'Null counts:\n')
display(amzn_df.null_count())
```

```
eda_amzn = amzn_df.clone()
eda_xlk = xlk_df.clone()

eda_cols = ['open', 'high', 'low', 'close', 'volume', 'trade_count', 'vwap', 'log_return',
            'hl_range', 'volume_ratio', 'rolling_vol', 'rolling_sharpe']

eda_amzn = eda_amzn.select(eda_cols)
eda_xlk = eda_xlk.select(eda_cols)

# distribution plot
pdf = eda_amzn.to_pandas()
scaled = StandardScaler().fit_transform(pdf[eda_cols])
scaled_df = pd.DataFrame(scaled, columns=eda_cols)

plt.figure(figsize=(10, 5))

scaled_df.boxplot(
    grid=False,
    flierprops=dict(marker='.', markersize=1, alpha=0.3)
)

plt.title("Feature Dist: AMZN (standardized)")
plt.xticks(rotation=15)
plt.ylim(-5, 5)
plt.show()

# outliers
z_scores = scaled_df.apply(zscore)
outlier_counts = (z_scores.abs() > 2).sum()
print(outlier_counts)


# target variable count
combined = amzn_df.with_columns(
    (pl.col("log_return").shift(-1) > 0).cast(pl.Int8).alias("target")
).drop_nulls()

print(combined["target"].value_counts())

# correlation plot
corr = amzn_df.select(eda_cols).to_pandas().corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, linewidths=0.5, square=True)
plt.title("AMZN feature correlation")
plt.tight_layout()
plt.show()

# log returns
plot_view_df = amzn_df.select(["timestamp", "log_return", "rolling_vol"]).to_pandas()
plot_view_df["timestamp"] = pd.to_datetime(plot_view_df["timestamp"])

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6), sharex=True)

ax1.plot(plot_view_df["timestamp"], plot_view_df["log_return"], lw=0.7)
ax1.axvspan(pd.Timestamp("2020-02-20"), pd.Timestamp("2020-04-01"), alpha=0.3, color="orange", label="COVID")
ax1.axvspan(pd.Timestamp("2022-01-01"), pd.Timestamp("2022-10-15"), alpha=0.12, color="blue", label="Rate hikes")
```

```
ax1.set_ylabel("log_return")
ax1.legend(fontsize=8)

ax2.plot(plot_view_df["timestamp"], plot_view_df["rolling_vol"], lw=0.8, color="steelblue")
ax2.set_ylabel("rolling_vol")
ax2.set_xlabel("date")

plt.suptitle("Log Returns Over Time", fontsize=10)
plt.tight_layout()
plt.show()


# split date decision
split_date = date(2025, 1, 1)

# selected features
features = ["log_return", "hl_range", "volume_ratio", "rolling_vol", "rolling_sharpe"]

# establishing labels based off of tomorrows log return
def target_label(df):

    df = (
        df.sort("timestamp").with_columns(
            (pl.col("log_return").shift(-1) > 0).cast(pl.Int32).alias("target")
        ).drop_nulls()
    )

    return df

# split definition to apply to both amzn and xlk
def split_df(df):

    temp_train =  df.filter(pl.col("timestamp") < split_date)
    x_train = temp_train[features].to_pandas()
    y_train = temp_train["target"].to_pandas()

    temp_test = df.filter(pl.col("timestamp") >= split_date)
    x_test = temp_test[features].to_pandas()
    y_test = temp_test["target"].to_pandas()

    return x_train, y_train, x_test, y_test

# run all steps
def perform_all(df):
    return split_df(target_label(df))


# store split results of both amzn and xlk with target defined
amzn_x_train, amzn_y_train, amzn_x_test, amzn_y_test = perform_all(amzn_df)
xlk_x_train, xlk_y_train, xlk_x_test, xlk_y_test = perform_all(xlk_df)
```

```
# mlflow
os.chdir("C:/Users/shure/TaroBall/Gtech/ISYE-7406-OAN/Project")
print(os.getcwd())
mlflow.set_tracking_uri("sqlite:///C:/Users/shure/TaroBall/Gtech/ISYE-7406-OAN/Project/data/mlflow/mlflow.db")
exp = mlflow.set_experiment("isye_hw5")
print(exp.artifact_location)

# helper def that helps with automating mlflow logging, runs and tests models as well
def mlflow_run_model(name, model, x_train, y_train, x_test, y_test, ticker, params=None):

    with mlflow.start_run(run_name=f"{name}_{ticker}"):

        # fit the model
        model.fit(x_train, y_train)
        pred = model.predict(x_test)

        # calculate scores
        acc = accuracy_score(y_test, pred)
        auc = roc_auc_score(y_test, model.predict_proba(x_test)[:,1])

        # log all results in ml_flow
        mlflow.log_params(params or {})
        mlflow.log_param("ticker", ticker)
        mlflow.log_metric("accuracy", acc)
        mlflow.log_metric("auc", auc)
        logged_model = mlflow.sklearn.log_model(model, name=name)

        # model evaluation
        eval_data = x_test.copy()
        eval_data["label"] = y_test.values
        mlflow.evaluate(
            model=logged_model.model_uri,
            data=eval_data,
            targets="label",
            model_type="classifier"
        )

        # physical prints
        print(f"Ticker: {ticker}")
        print(f"Name: {name}")
        print(f"Acc: {acc}")
        print(f"AUC: {auc}")
        print(confusion_matrix(y_test, pred), "\n")

# loop through all models and tickers at once
loop_sequence = [
    ("AMZN", amzn_x_train, amzn_y_train, amzn_x_test, amzn_y_test),
    ("XLK",  xlk_x_train,  xlk_y_train, xlk_x_test, xlk_y_test),
]

for ticker, temp_x_train, temp_y_train, temp_x_test, temp_y_test in loop_sequence:

    # logistic regression: baseline model with no tuning
    mlflow_run_model(
        "LogisticRegression",
        LogisticRegression(max_iter=1000),
        temp_x_train,
        temp_y_train,
        temp_x_test,
        temp_y_test,
        ticker
    )
```

```
# decision tree: baseline model with no tuning
mlflow_run_model(
    "DecisionTree",
    DecisionTreeClassifier(max_depth=5, random_state=42),
    temp_x_train,
    temp_y_train,
    temp_x_test,
    temp_y_test,
    ticker
)

# setting tcsv
tscv = TimeSeriesSplit(n_splits=5)

# random forest with grid serach cv
rf_params = {
    "n_estimators": [100, 200],
    "max_depth": [3, 5, None],
    "max_features": ["sqrt", "log2"]
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=420),
    rf_params,
    cv=tscv,
    scoring="roc_auc",
    n_jobs=-1
)

rf_grid.fit(temp_x_train, temp_y_train)
mlflow_run_model(
    "RandomForest",
    rf_grid.best_estimator_,
    temp_x_train,
    temp_y_train,
    temp_x_test,
    temp_y_test,
    ticker,
    rf_grid.best_params_
)

# xgboost with grids search cv

xgb_params = {
    "n_estimators": [100, 200],
    "max_depth": [3, 5],
    "learning_rate": [0.01, 0.1]
}

xgb_grid = GridSearchCV(
    xgb.XGBClassifier(eval_metric="logloss", random_state=420),
    xgb_params,
    cv=tscv,
    scoring="roc_auc",
    n_jobs=-1
)

xgb_grid.fit(temp_x_train, temp_y_train)

mlflow_run_model(
    "XGBoost",
    xgb_grid.best_estimator_,
    temp_x_train,
    temp_y_train,
```